

رسالة محمد



# یادگیری ماشین

مدرس: محمدرضا محمدی

زمستان ۱۴۰۱

دسته‌بندی

Classification

# مجموعه داده MNIST

- در این فصل از مجموعه داده MNIST استفاده می‌کنیم که شامل ۷۰۰۰۰ نمونه تصویر کوچک از رقم‌های دستنویس است



```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)

>>> X, y = mnist.data, mnist.target

>>> X.shape
(70000, 784)

>>> y.shape
(70000,)

>>> y
array(['5', '0', '4', ..., '4', '5', '6'], dtype=object)
```

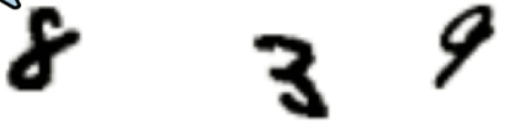
# ماتریس درهم ریختگی (Confusion)

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

• صحت (precision): دقت پیش بینی های مثبت

• پوشش یا بازیابی (recall): درصد نمونه های مثبتی که به درستی دسته بندی شده است

|        |          | Predicted                                                                                        |                                                                                                    |
|--------|----------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
|        |          | Negative                                                                                         | Positive                                                                                           |
| Actual | Negative | <div>TN</div>  | <div>FP</div>   |
|        | Positive |                | <div>TP</div>  |
|        |          | <div>FN</div>                                                                                    | <div>Precision (e.g., 3 out of 4)</div>                                                            |
|        |          | <div>Recall (e.g., 3 out of 5)</div>                                                             |                                                                                                    |

|              |          | Predicted Class |          |
|--------------|----------|-----------------|----------|
|              |          | Negative        | Positive |
| Actual Class | Negative | TN              | FP       |
|              | Positive | FN              | TP       |

```
array([[53892, 687],
       [1891, 3530]])
```

# معیارهای ارزیابی

```
>>> from sklearn.metrics import precision_score, recall_score
>>> precision_score(y_train_5, y_train_pred) # == 3530 / (687 + 3530)
0.8370879772350012
>>> recall_score(y_train_5, y_train_pred) # == 3530 / (1891 + 3530)
0.6511713705958311
```

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

```
>>> from sklearn.metrics import f1_score
>>> f1_score(y_train_5, y_train_pred)
0.7325171197343846
```

```
array([[53892, 687],
       [1891, 3530]])
```

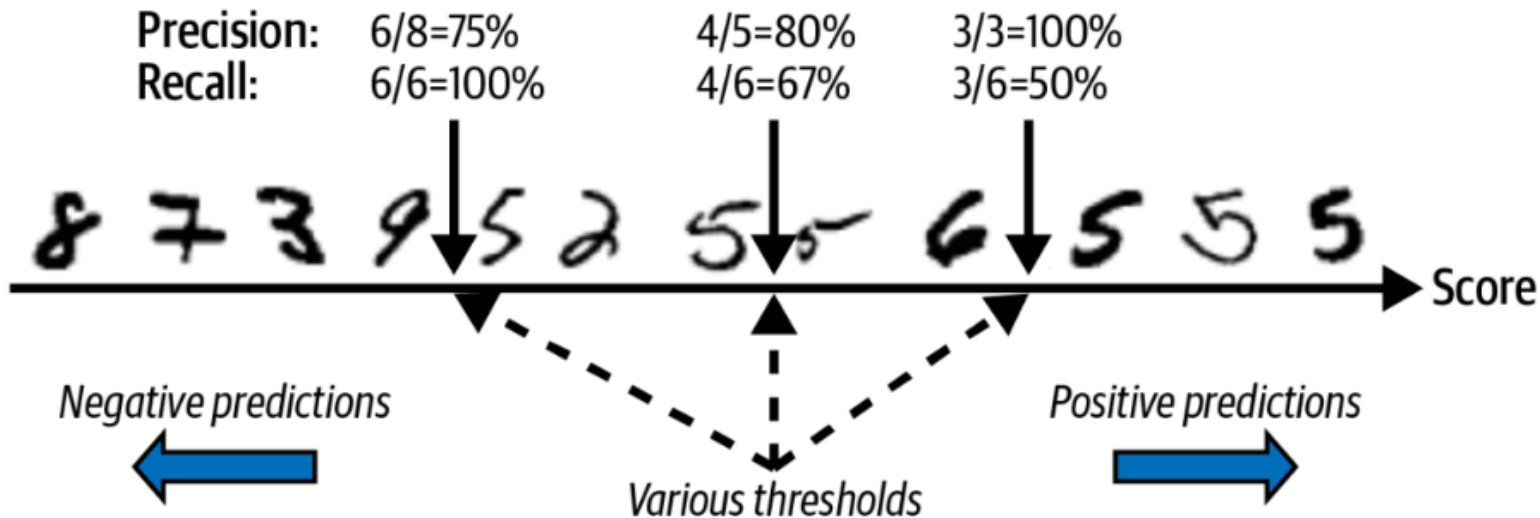
- هر کدام از این دو معیار فقط از یک زاویه به عملکرد مدل نگاه می‌کنند

- می‌توان مقادیر آنها را ترکیب کرد تا بتوان با یک مقدار عددی، الگوریتم‌های مختلف را مقایسه کرد

- بسته به مسئله، می‌توان اهمیت precision یا recall را افزایش داد

# Precision/Recall Trade-off

- دسته‌بندی متداول، برای هر نمونه یک امتیاز (score) بر اساس تابع تصمیم (decision function) آموخته شده پیش‌بینی می‌کند
  - اگر امتیاز بیش از یک حد آستانه (threshold) باشد، به عنوان دسته مثبت در نظر گرفته می‌شود
  - حد آستانه را چند قرار دهیم؟

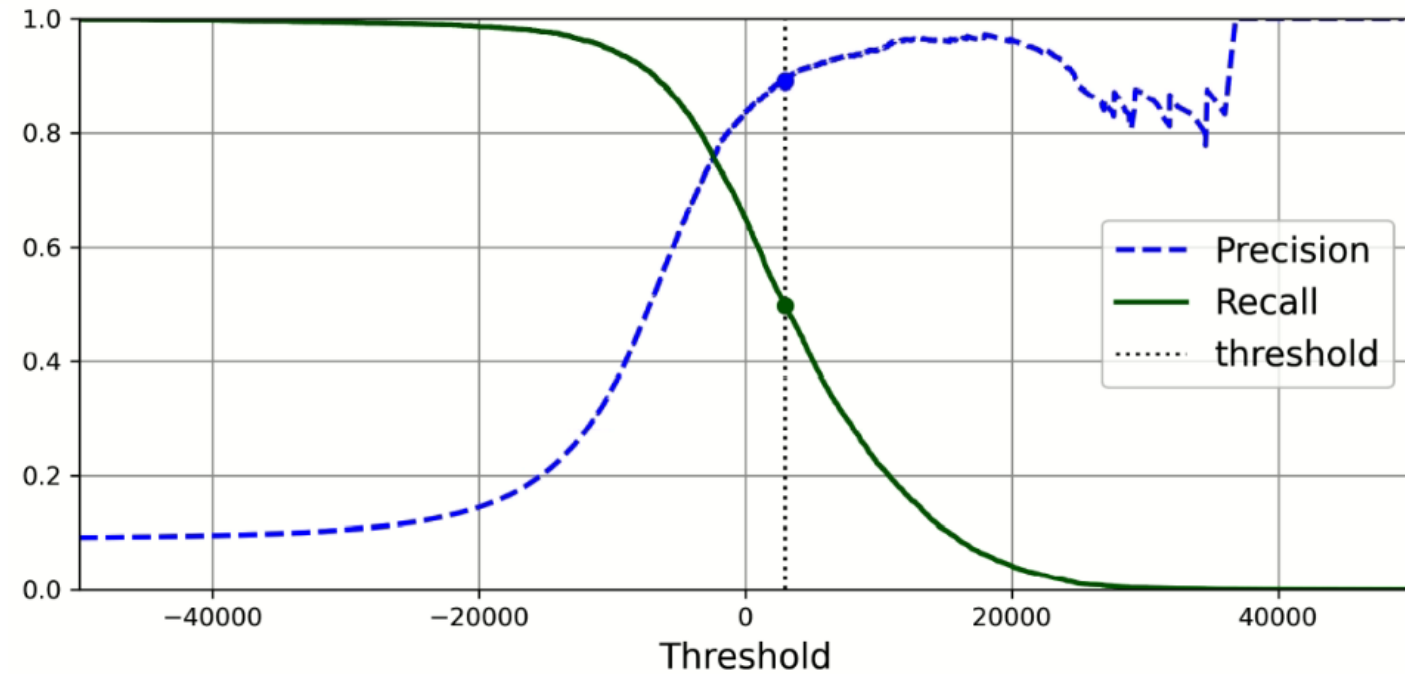


```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3,  
                             method="decision_function")
```

```
from sklearn.metrics import precision_recall_curve
```

```
precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
```

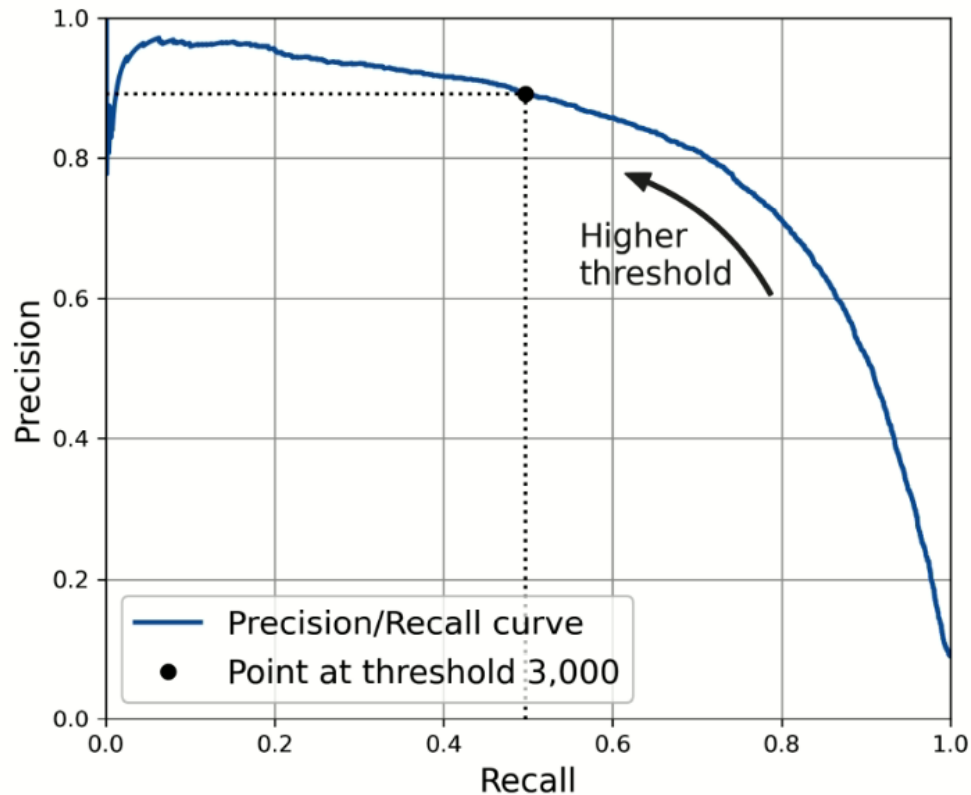
```
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)  
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)  
plt.vlines(threshold, 0, 1.0, "k", "dotted", label="threshold")  
[...] # beautify the figure: add grid, legend, axis, labels, and circles  
plt.show()
```





# منحنی Precision-Recall

```
plt.plot(recalls, precisions, linewidth=2, label="Precision/Recall curve")  
[...] # beautify the figure: add labels, grid, legend, arrow, and text  
plt.show()
```



- فرض کنید نیاز به سیستمی داریم که حداقل دقت آن برابر با 90% باشد

```
>>> idx_for_90_precision = (precisions >= 0.90).argmax()  
>>> threshold_for_90_precision = thresholds[idx_for_90_precision]  
>>> threshold_for_90_precision  
3370.0194991439557  
  
y_train_pred_90 = (y_scores >= threshold_for_90_precision)  
  
>>> precision_score(y_train_5, y_train_pred_90)  
0.9000345901072293  
>>> recall_at_90_precision = recall_score(y_train_5, y_train_pred_90)  
>>> recall_at_90_precision  
0.4799852425751706
```

# منحنی ROC

- منحنی receiver operating characteristic (ROC) یکی دیگر از ابزارهای متداول برای ارزیابی مدل‌های دسته‌بندی باینری است

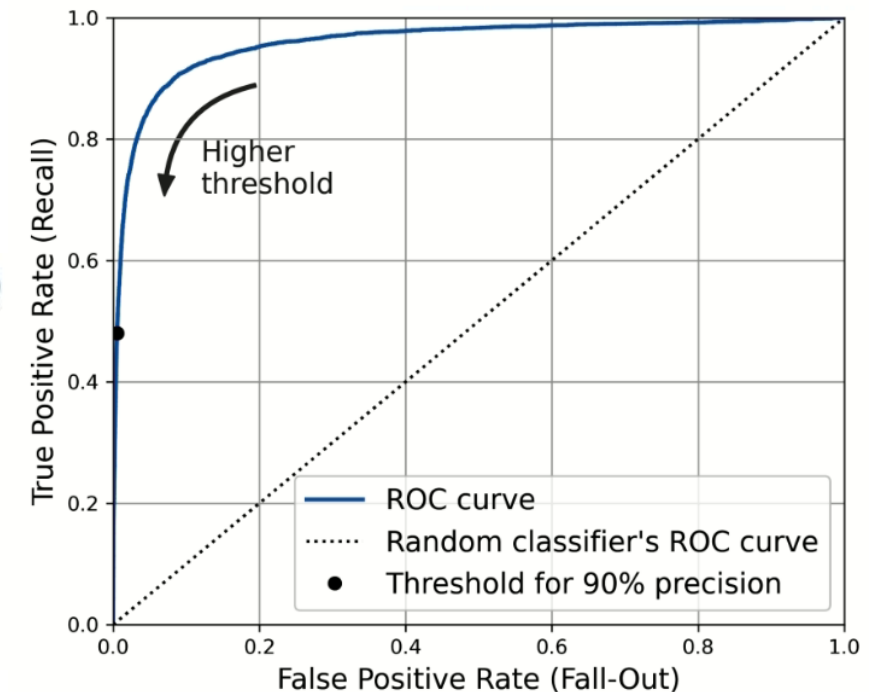
- بسیار شبیه به منحنی precision/recall است اما TPR (recall) را بر حسب FPR (1-specificity) رسم می‌کند

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)

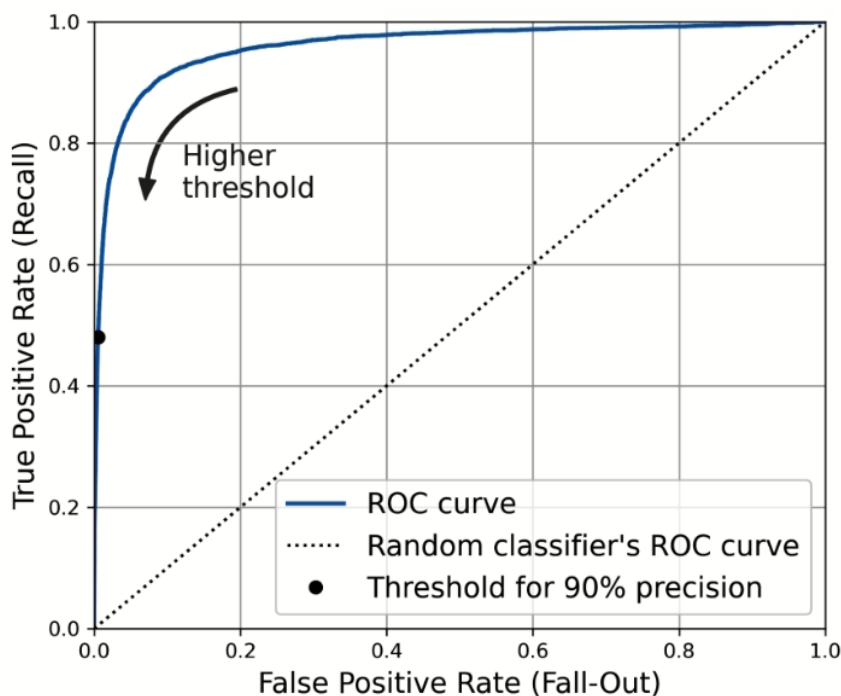
idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax()
tpr_90, fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]

plt.plot(fpr, tpr, linewidth=2, label="ROC curve")
plt.plot([0, 1], [0, 1], 'k:', label="Random classifier's ROC curve")
plt.plot([fpr_90], [tpr_90], "ko", label="Threshold for 90% precision")
[...] # beautify the figure: add labels, grid, legend, arrow, and text
plt.show()
```



# منحنی ROC

- خط نقطه چین منحنی یک دسته بندی کاملاً تصادفی را نشان می دهد
- یک دسته بند خوب باید تا حد امکان از آن دور باشد (به سمت گوشه سمت چپ بالا)
- یکی راه برای مقایسه دسته بندها، اندازه گیری سطح زیر منحنی (AUC) است



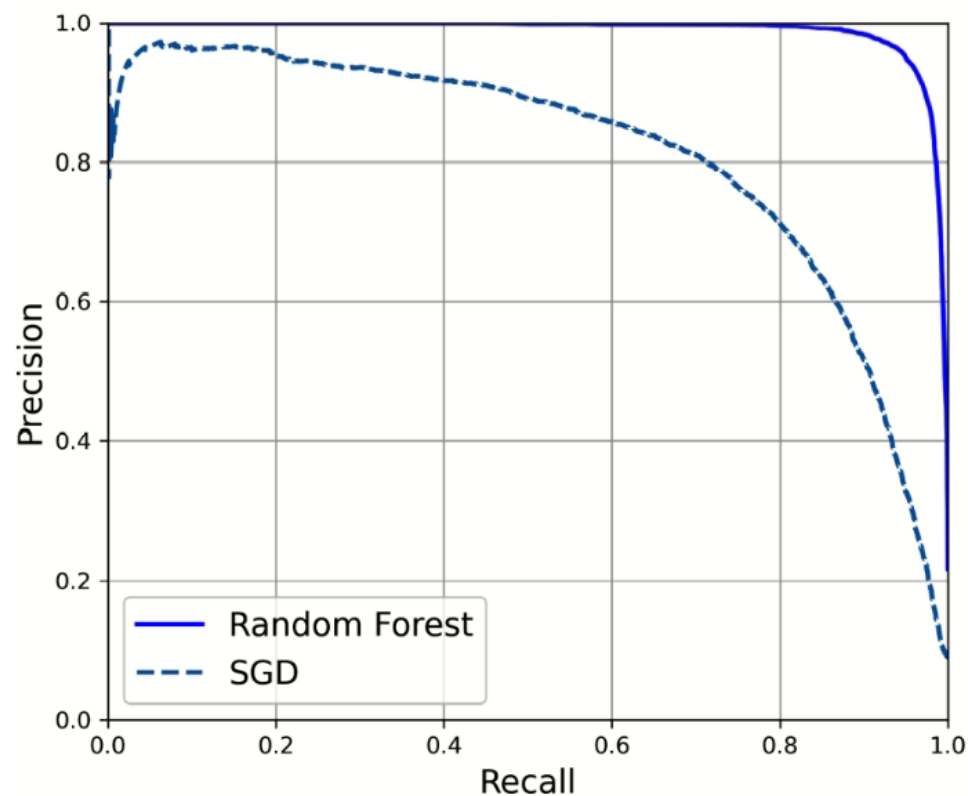
- یک دسته بند ایده آل دارای ROC AUC برابر با ۱ خواهد بود
- یک دسته بند کاملاً تصادفی دارای ROC AUC برابر با ۰.۵ خواهد بود

```
>>> from sklearn.metrics import roc_auc_score  
>>> roc_auc_score(y_train_5, y_scores)  
0.9604938554008616
```

- بسته به مسئله می توان از منحنی ROC یا PR استفاده کرد

# دسته‌بند RandomForest

- برای مقایسه با SGDClassifier از RandomForestClassifier استفاده می‌کنیم



```
>>> y_train_pred_forest = y_probas_forest[:, 1] >= 0.5  
>>> f1_score(y_train_5, y_pred_forest)  
0.9242275142688446  
>>> roc_auc_score(y_train_5, y_scores_forest)  
0.9983436731328145
```

# دسته‌بندی چند کلاسه

- بسیاری از مدل‌های یادگیری ماشین به صورت ذاتی می‌توانند این کار را انجام بدهند
- اما برخی مدل‌های یادگیری ماشین مانند SVC تنها توانایی دسته‌بندی دو کلاسه را دارند

- می‌توان برای شناسایی هر کلاس یک دسته‌بند جداگانه طراحی کرد

▪ به این رویکرد one-versus-the-rest (OvR) یا one-versus-all (OvA) گفته می‌شود

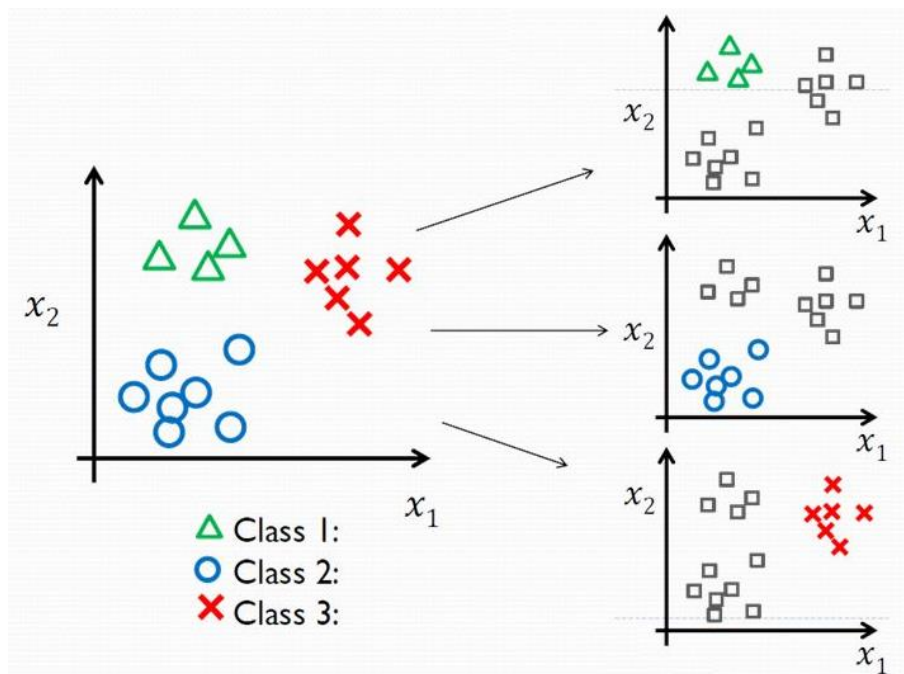
- می‌توان برای جداسازی هر دو کلاس یک دسته‌بند جداگانه طراحی شود

▪ به این رویکرد one-versus-one (OvO) گفته می‌شود

▪ به تعداد  $N \times (N - 1) / 2$  دسته‌بند نیاز است

▪ کلاسی در نهایت انتخاب می‌شود که دفعات بیشتری بنده شود

- تعداد دسته‌بندهای لازم برای OvO می‌تواند خیلی زیاد شود اما هر دسته‌بند بر روی داده‌های کمتر و متوازن‌تری آموزش می‌بیند





# دسته‌بندی چند کلاسه

- بسیاری از مدل‌های موجود، به صورت خودکار از OvO یا OVR برای تبدیل یک دسته‌بند دو کلاسه به یک دسته‌بند چند کلاسه استفاده می‌کنند

- در این مثال ۴۵ دسته‌بند دو کلاسه آموزش داده خواهد شد

```
from sklearn.svm import SVC
```

```
svm_clf = SVC(random_state=42)  
svm_clf.fit(X_train[:2000], y_train[:2000])
```

```
>>> svm_clf.predict([some_digit])  
array(['5'], dtype=object)
```

```
>>> some_digit_scores = svm_clf.decision_function([some_digit])  
>>> some_digit_scores.round(2)  
array([[ 3.79,  0.73,  6.06,  8.3 , -0.29,  9.3 ,  1.75,  2.77,  7.21,  
         4.82]])
```

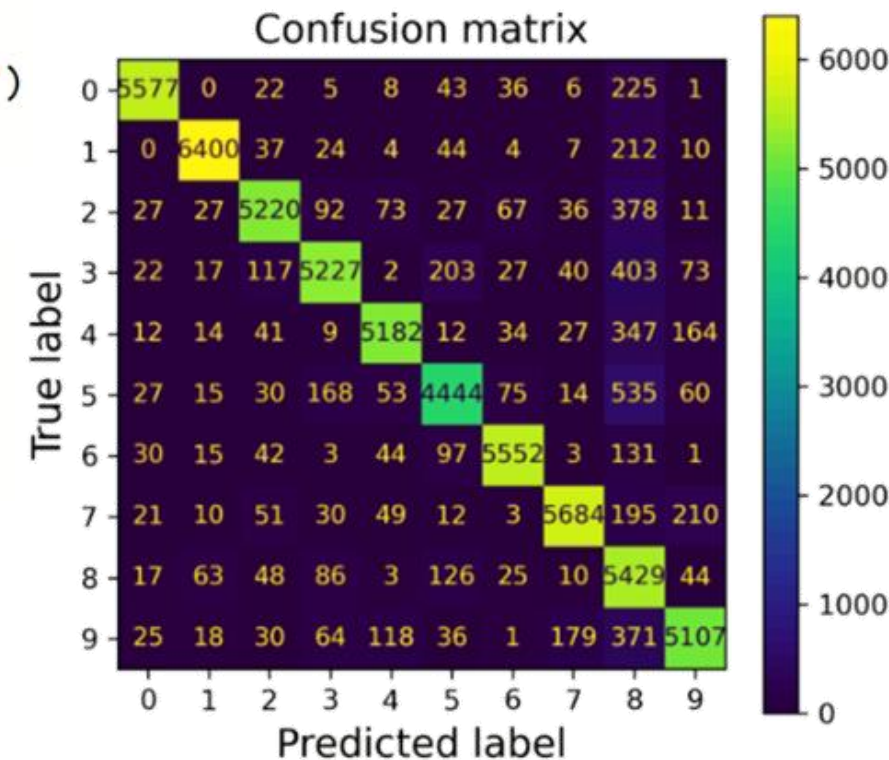
# تحلیل خطا

- ماتریس درهم‌ریختگی یکی از روش‌های ساده و پرکاربرد برای تحلیل خطا است

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler()
>>> X_train_scaled = scaler.fit_transform(X_train.astype("float64"))
>>> cross_val_score(sgd_clf, X_train_scaled, y_train, cv=3, scoring="accuracy")
array([0.8983, 0.891 , 0.9018])
```

```
from sklearn.metrics import ConfusionMatrixDisplay
```

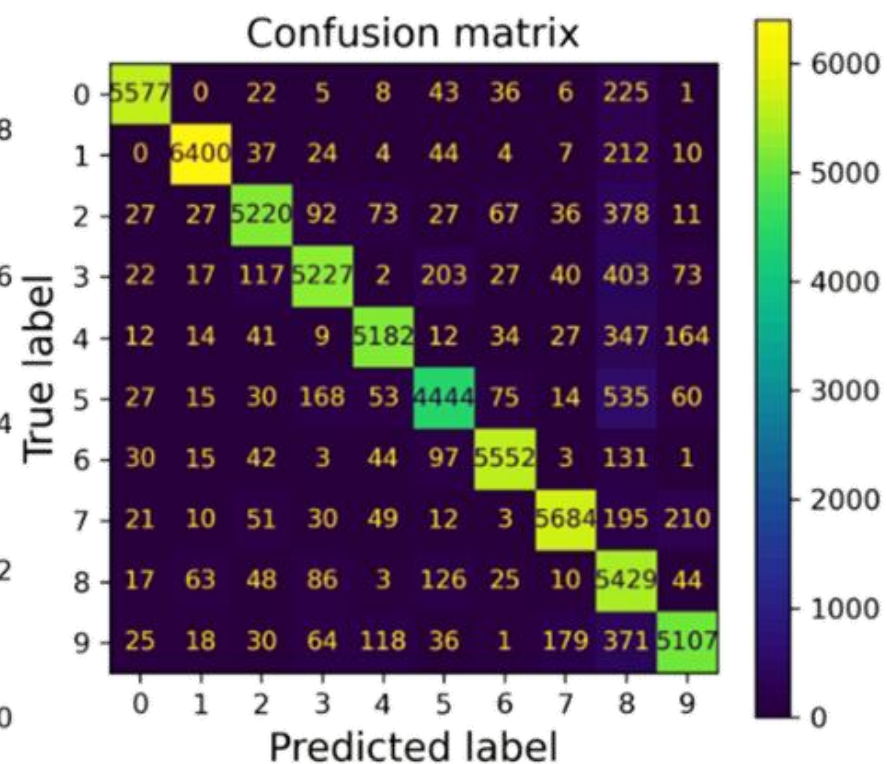
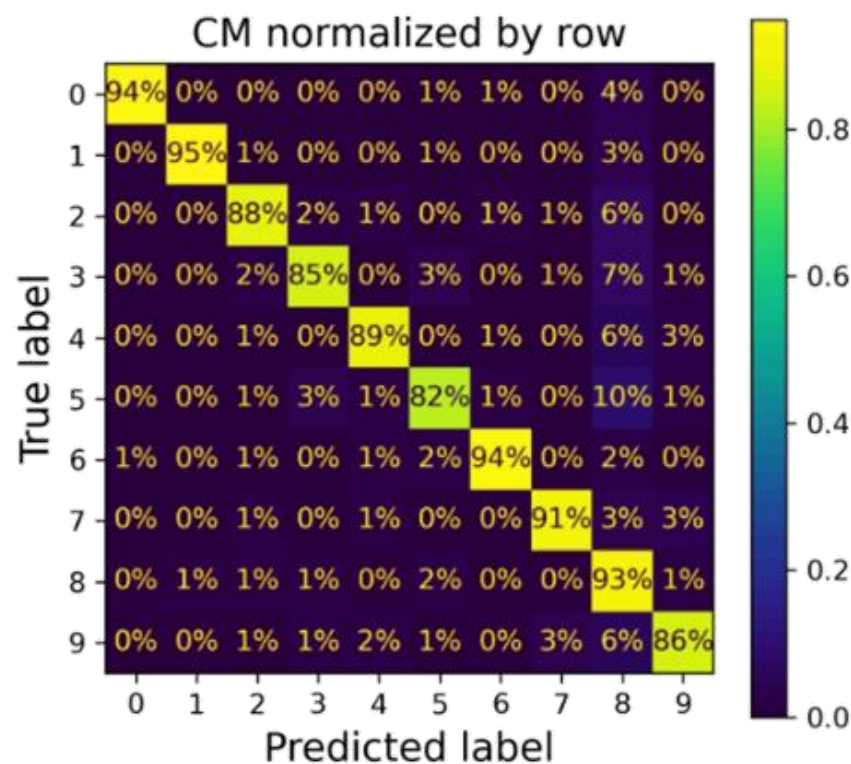
```
y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)
plt.show()
```



```
from sklearn.metrics import ConfusionMatrixDisplay
```

```
y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)  
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)  
plt.show()
```

```
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,  
                                       normalize="true", values_format=".0%")  
plt.show()
```



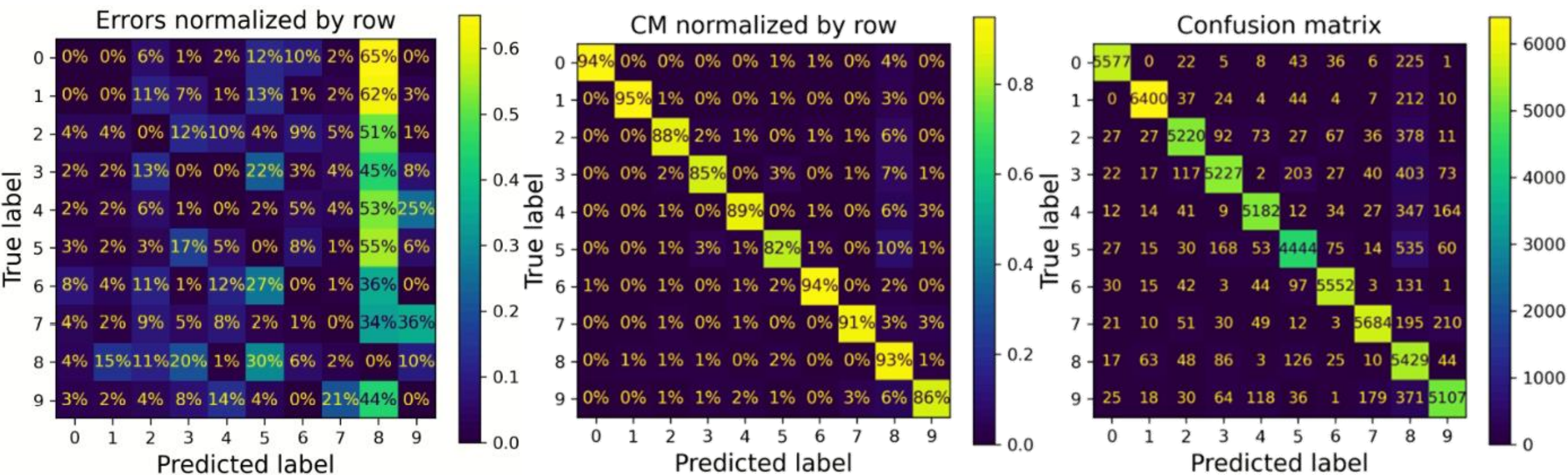


```

sample_weight = (y_train_pred != y_train)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                       sample_weight=sample_weight,
                                       normalize="true", values_format=".0%")

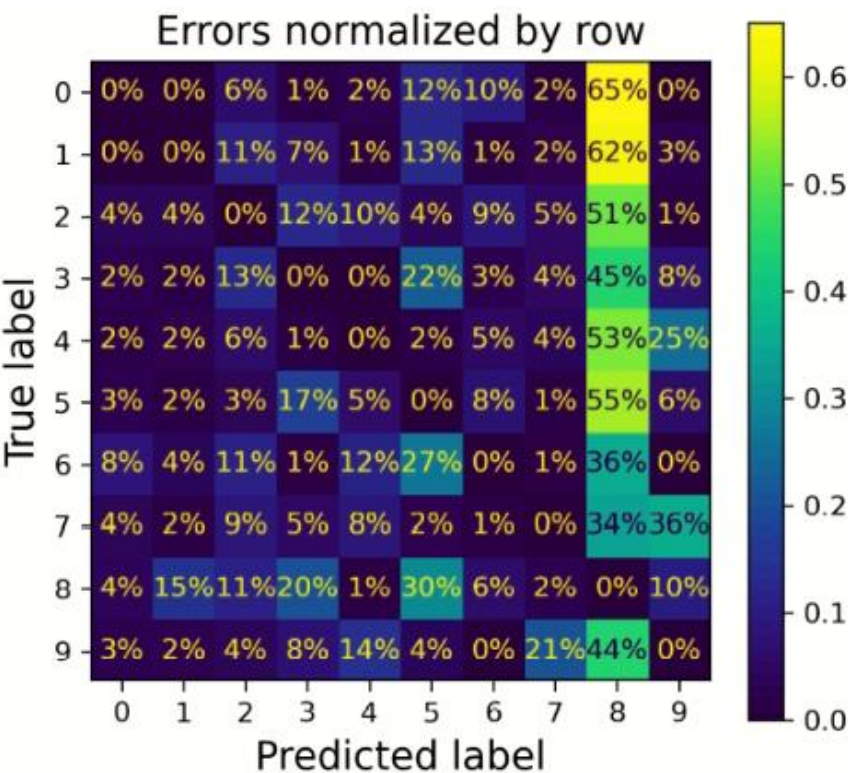
plt.show()

```



# تحلیل خطا

- تحلیل ماتریس درهم‌ریختگی در بسیاری از مواقع می‌تواند برای بهبود عملکرد دسته‌بند مناسب باشد
  - در این مثال، خوب است تا برای کاهش خطای 8 تلاش کنیم

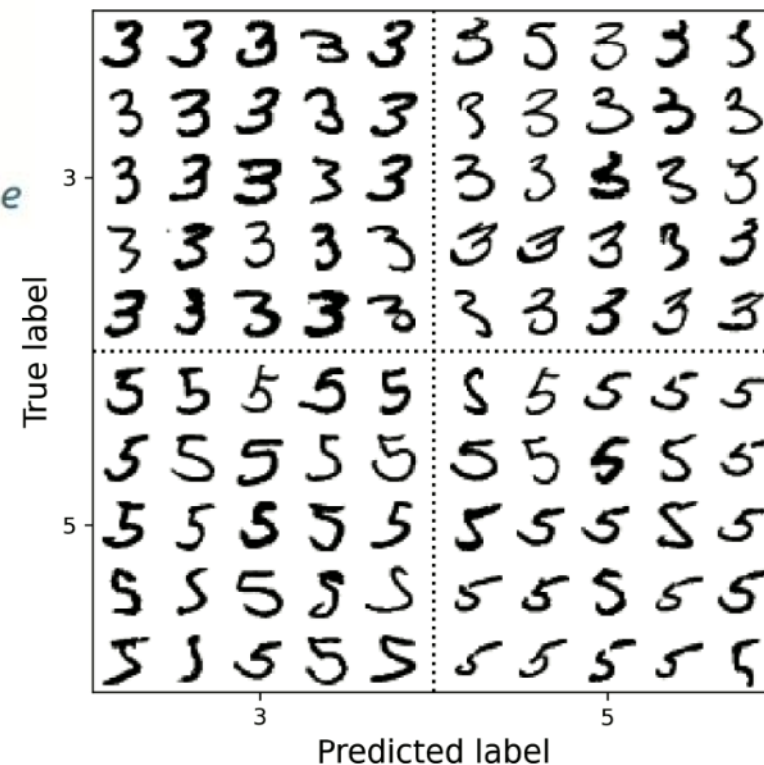


- داده‌های بیشتری از کلاس‌های دیگر که شبیه به 8 باشند جمع کنیم
- ویژگی‌های جدیدی استخراج کنیم (مثلا تعداد حلقه‌های بسته)
- پیش‌پردازش‌های بهتری انجام دهیم

# تحلیل خطا

- نمایش نمونه‌هایی که دچار اشتباه شده‌اند هم می‌تواند خیلی موثر باشد

```
cl_a, cl_b = '3', '5'  
X_aa = X_train[(y_train == cl_a) & (y_train_pred == cl_a)]  
X_ab = X_train[(y_train == cl_a) & (y_train_pred == cl_b)]  
X_ba = X_train[(y_train == cl_b) & (y_train_pred == cl_a)]  
X_bb = X_train[(y_train == cl_b) & (y_train_pred == cl_b)]  
[...] # plot all images in X_aa, X_ab, X_ba, X_bb in a confusion matrix style
```





# تحلیل خطا

- برخی نمونه‌های خطا واقعا بدخط نوشته شده‌اند اما برخی نمونه‌ها واضح هستند

- ممکن است خطای برچسب‌گذاری هم مشاهده شود

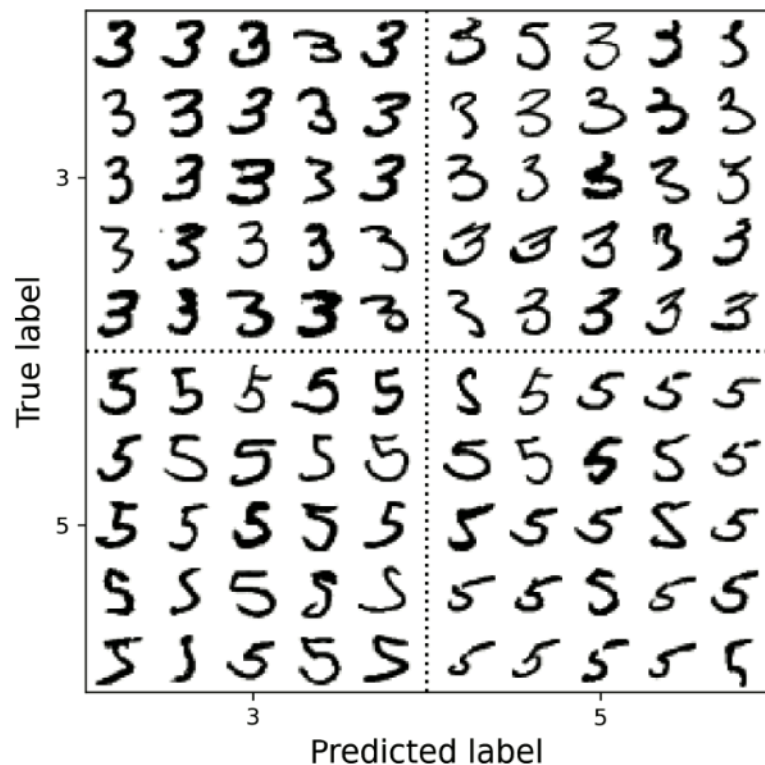
- پیدا کردن علت خطای دسته‌بند کار ساده‌ای نیست

- در این مثال از یک دسته‌بند خطی ساده استفاده کرده بودیم

- نسبت به موقعیت پیکسل‌ها هم حساس است و جابجایی باعث خطا می‌شود

- می‌توان تلاش کرد تا کاراکتر به وسط تصویر منتقل و زاویه آن جبران شود

- می‌توان از داده‌افزایی (data augmentation) استفاده کرد



# دسته‌بندی چندبرچسبه (Multilabel)



Face Recognition

Facial Expression Recognition

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
```

```
y_train_large = (y_train >= '7')
y_train_odd = (y_train.astype('int8') % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]
```

```
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
```

```
>>> knn_clf.predict([some_digit])
array([[False,  True]])
```

- در برخی مسائل نیاز داریم که دسته‌بند بتواند داده ورودی را از چند جنبه دسته‌بندی کند
- به عنوان یک مثال شبیه‌سازی فرض کنید بخواهیم از روی تصویر یک رقم دستنویس، بزرگ بودن و فرد بودن عدد را تشخیص بدهیم
- برخی از مدل‌های یادگیری ماشین حالت چند خروجی را پشتیبانی می‌کنند



آموزش مدل‌ها

Training Models

# رگرسیون خطی

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x} = \boldsymbol{\theta}^T \mathbf{x}$$

- این معادله را می‌توان به صورت برداری نوشت (ضرب داخلی)
- $\boldsymbol{\theta}$  بردار پارامترهای مدل است (شامل بایاس  $\theta_0$ )
- $\mathbf{x}$  بردار ویژگی‌های نمونه است (شامل مقدار ۱ به عنوان  $x_0$ )
- $m$  معادله و  $n + 1$  مجهول داریم که معمولاً  $m \gg n$  است
- برای آموزش مدل نیاز است تا یک تابع ضرر تعریف کنیم
  - میانگین مربعات خطا تابع مناسبی است

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m \left( \boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

# رگرسیون خطی

- حال باید  $\theta$  بهینه را بدست بیاوریم که منجر به کمترین خطا می شود
- $MSE(\mathbf{X}, h_{\theta}) = \frac{1}{m} \sum_{i=1}^m (\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$
- یک راه برای بهینه سازی این است که مشتق تابع را محاسبه کرده و برابر با صفر قرار دهیم

$$\frac{\partial MSE(\mathbf{X}, h_{\theta})}{\partial \theta} = \frac{1}{m} \sum_{i=1}^m 2(\theta^T \mathbf{x}^{(i)} - y^{(i)}) \mathbf{x}^{(i)} = \frac{1}{m} \sum_{i=1}^m 2 \mathbf{x}^{(i)} (\mathbf{x}^{(i)T} \theta - y^{(i)}) = 0$$

$$\Rightarrow \sum_{i=1}^m \mathbf{x}^{(i)} \mathbf{x}^{(i)T} \hat{\theta} = \sum_{i=1}^m \mathbf{x}^{(i)} y^{(i)} \Rightarrow \mathbf{X}^T \mathbf{X} \hat{\theta} = \mathbf{X}^T \mathbf{y} \Rightarrow \hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \mathbf{X}^{\dagger} \mathbf{y}$$

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(1)T} \\ \vdots \\ \mathbf{x}^{(m)T} \end{bmatrix}$$

- خوشبختانه این مسئله بهینه سازی دارای پاسخ بسته است
- به آن معادله نرمال گفته می شود